

MAC security (unforgeability)

Experiment $\text{MAC}_{A,\Pi}(n)$:

1. A key k is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , and **oracle access to $\text{MAC}_k(\cdot)$** . Let Q be the set of all queries made by A
3. A outputs (m, t)
4. Experiment outputs 1 if (1) $\text{verify}_k(m, t) = 1$ and (2) $m \notin Q$

Definition: a Mac $\Pi = (\text{Gen}, \text{MAC}, \text{Verify})$ is **unforgeable/secure**, if for all probabilistic polynomial time adversaries A , there is a negligible function negl such that for all n ,

$$\Pr[\text{MAC}_{A,\Pi}(n) = 1] \leq \text{negl}(n)$$

Construction 1: fixed-length MAC

Let F be a PRF

- $k \leftarrow \text{Gen}(1^n)$
- $\text{MAC}_k(m): t = F_k(m)$
- $\text{Verify}_k(m, t):$ compute $F_k(m)$, check if $t = F_k(m)$

Longer messages?

Idea 4: parse m as $m_1, m_2, \dots, m_l$, compute $t_i = F_k(r || l || i || m_i)$

Send $(r, t_1, \dots, t_l)$ as the tag

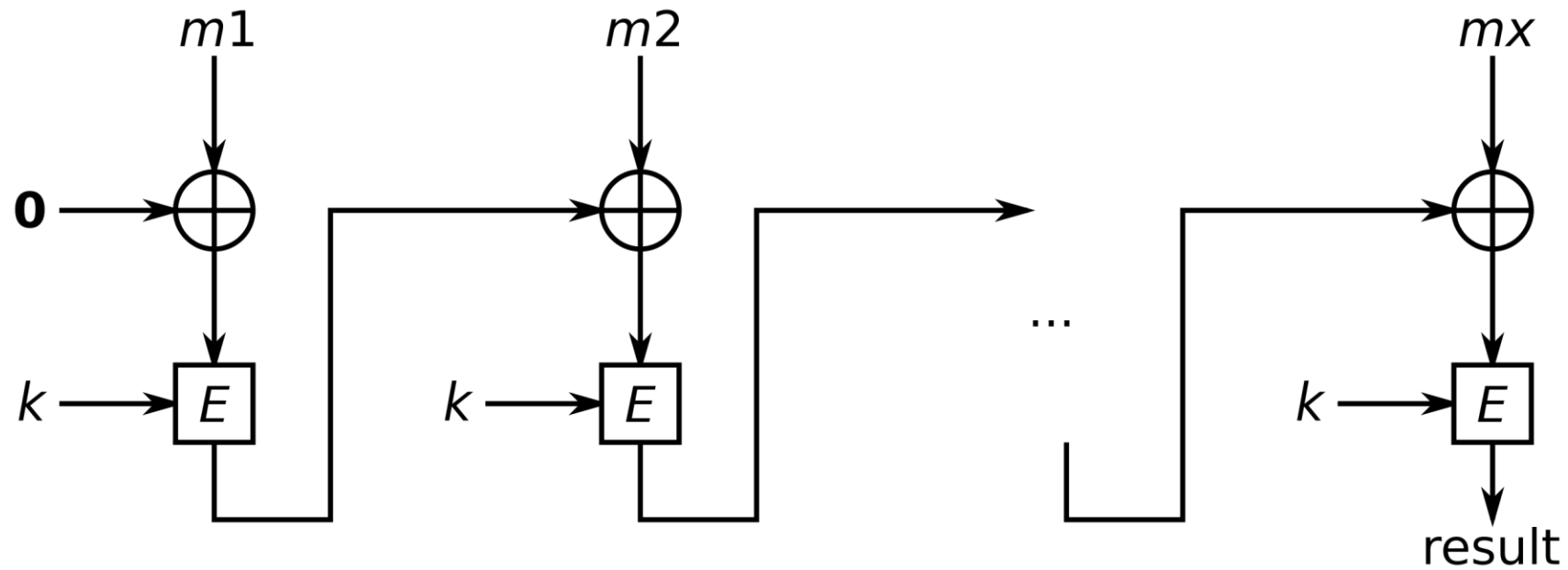
It is a secure tag. Proof.

Two events: Repeat r or a new block contained in Adv's message

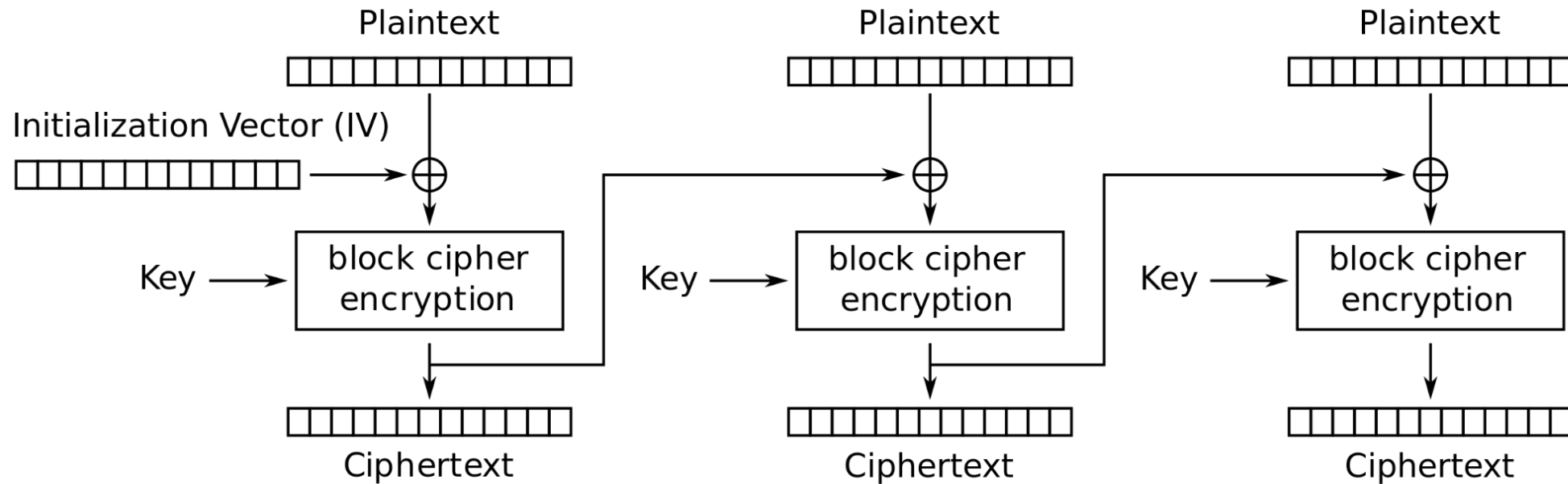
MAC in practice

- Can be deterministic
- Can be shorter than the message

CBC-MAC



CBC-mode encryption

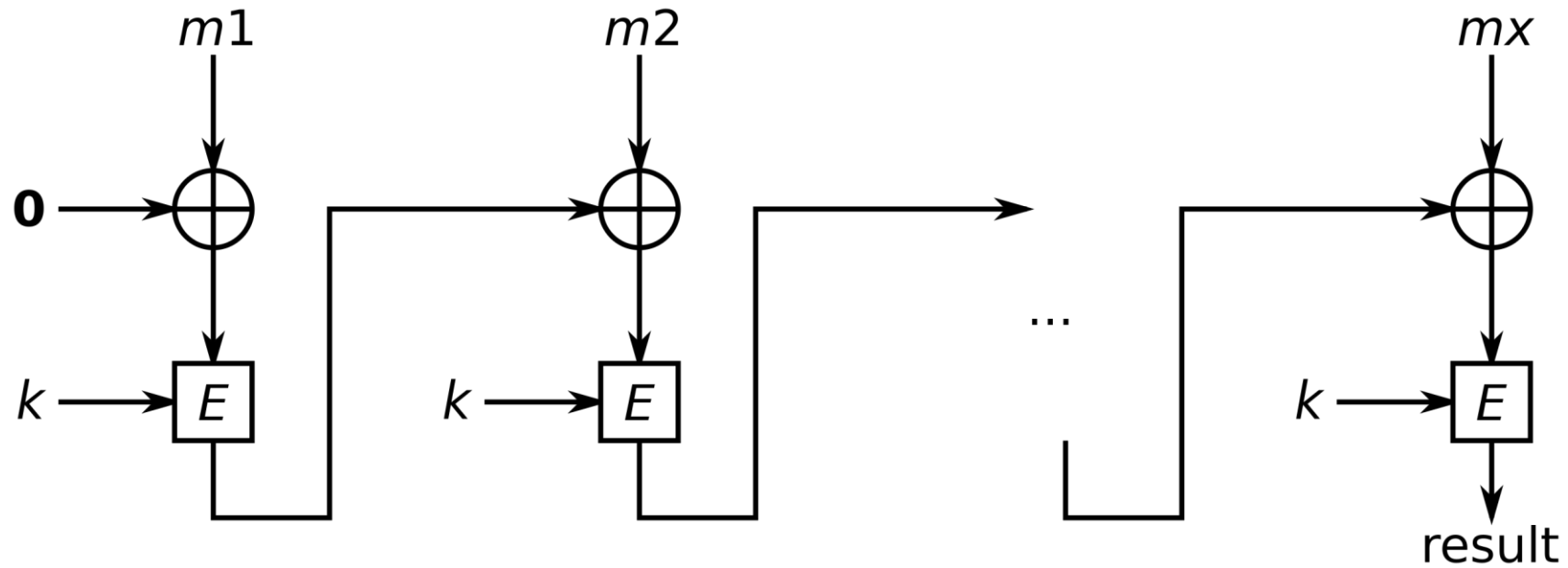


Cipher Block Chaining (CBC) mode encryption

CBC MAC vs. CBC encryption

1. CBC MAC is deterministic, IV has to be 0
 - Not secure if IV is randomized, why?
2. t is shorter than the message
 - Not secure if we include the output of every block cipher encryption

CBC-MAC

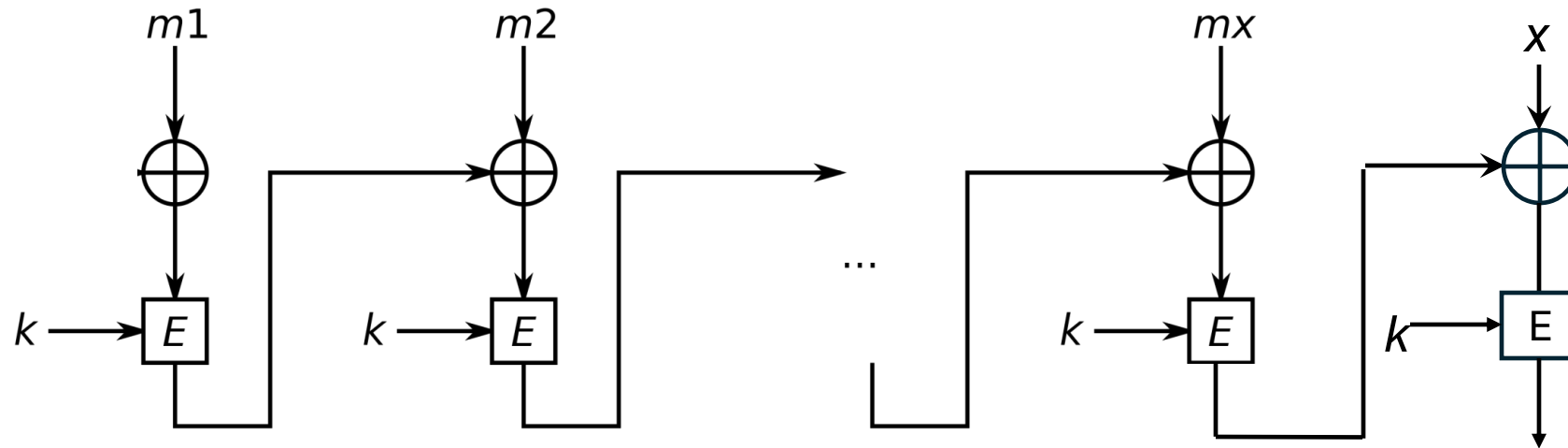


Still only secure for fixed length message!!

Length extension attack

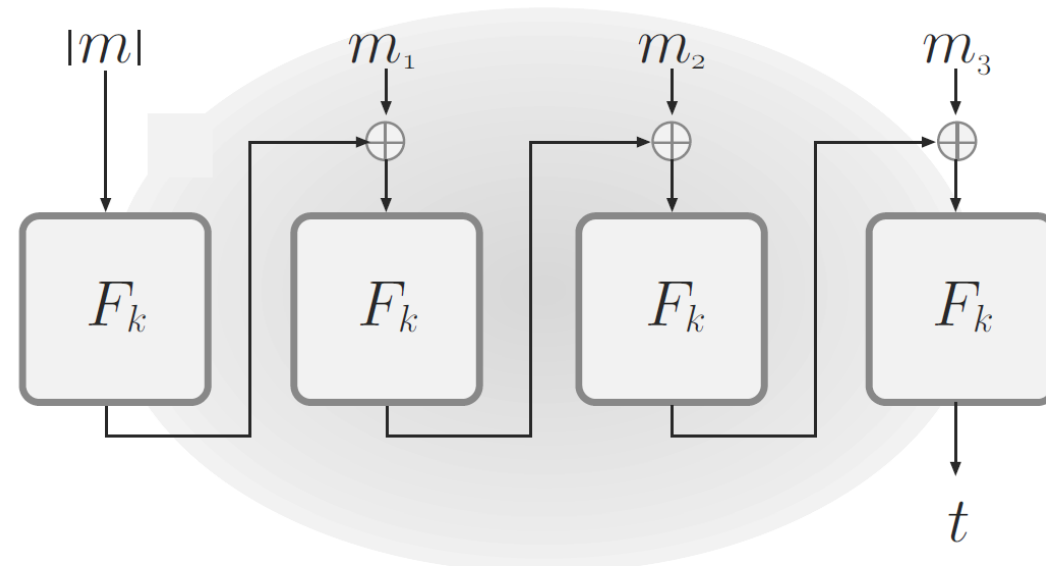
CBC-MAC for arbitrary-length messages

1. Append the message length $|m|$ as an n -bit string



CBC-MAC for arbitrary-length messages

1. Prepend the message length $|m|$ as an n-bit string



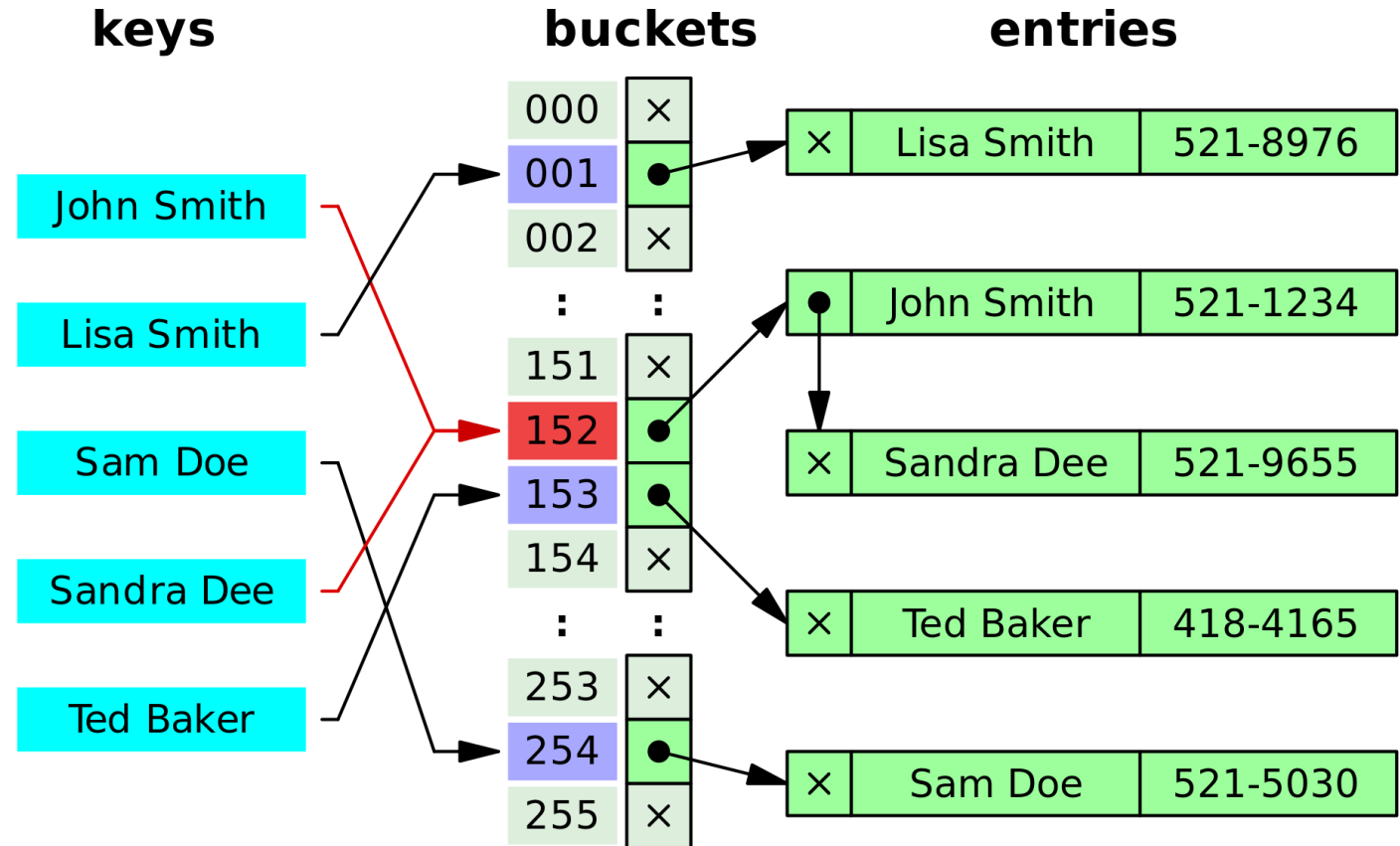
Cryptographic Hash Functions

Hash Functions

- A **hash function** H maps an input x of **arbitrary length** to a **fixed-length** value $y = H(x)$. Hash value or digest of x
 - A **collision** is a pair of plaintexts x and x' that map to the same hash value,
 $H(x) = H(x')$
 - **Collisions are unavoidable** if the input size is larger than the output size

Hash table

- Search data structure. Storing items in locations associated with their hash value
- Chaining for collisions
- The hash function should spread input uniformly for fast search time



Cryptographic hash functions

- Maps arbitrary length input into finite length output $H: \{0,1\}^* \rightarrow \{0,1\}^l$
- Properties of a secure cryptographic hash function H
 - **Collision resistance**
Computationally **infeasible** to find any pair (x, x') such that $H(x) = H(x')$
 - **One-way** (easy to compute, hard to invert)
Given $y = H(x)$, computationally **infeasible** to find x

Applications of cryptographic hash function

- Data integrity (no key, software checksum, collision resistant)
e.g., <https://www.virtualbox.org/wiki/Downloads>
- Storing passwords (one-way and collision resistant)



IMAGE: DOLE777 VIA UNSPLASH

Alexander Martin

September 27th, 2024

Industry

Technology

News

**Meta fined \$101 million for storing
hundreds of millions of passwords in
plaintext**

Definition

A hash function is a pair of polynomial-time algorithms (Gen, H) :

- Gen is a probabilistic algorithm that takes a security parameter 1^n and outputs a seed s
 - Why seed s ?
- H is a deterministic algorithm that takes a seed s and a string $x \in \{0,1\}^*$ and outputs $H^s(x) \in \{0,1\}^l$

$H(x)$ for short.

If H is defined only for x of length $l' > l$, we say that it is a fixed-length hash function for l' , and it is a **compression** function.

Collision resistance

Experiment $\text{col}_{A,\Pi}(n)$:

1. A seed s is generated by $\text{Gen}(1^n)$
2. The adversary A is given s , and outputs $x, x' (\in \{0,1\}^{l'})$ if fixed-length
3. If $H^s(x) = H^s(x')$ and $x \neq x'$, A has found a collision and the experiment outputs 1

A hash function (Gen, H) is collision resistant if for all PPT A , there is a negl such that

$$\Pr[\text{col}_{A,\Pi}(n)=1] \leq \text{negl}(n)$$

One-way function

- Sufficient and necessary for almost all symmetric-key primitives
- One-wayness does not imply collision resistance
- Collision resistance does not imply one-wayness

Some properties of hash function

- “ratio of shrinking”

$$H: \{0,1\}^{512} \rightarrow \{0,1\}^{256}$$

$$H: \{0,1\}^{1024} \rightarrow \{0,1\}^{256}$$

- Hardness of finding a collision

- $H: \{0,1\}^{512} \rightarrow \{0,1\}^{256}$

- $H: \{0,1\}^{1024} \rightarrow \{0,1\}^{256}$

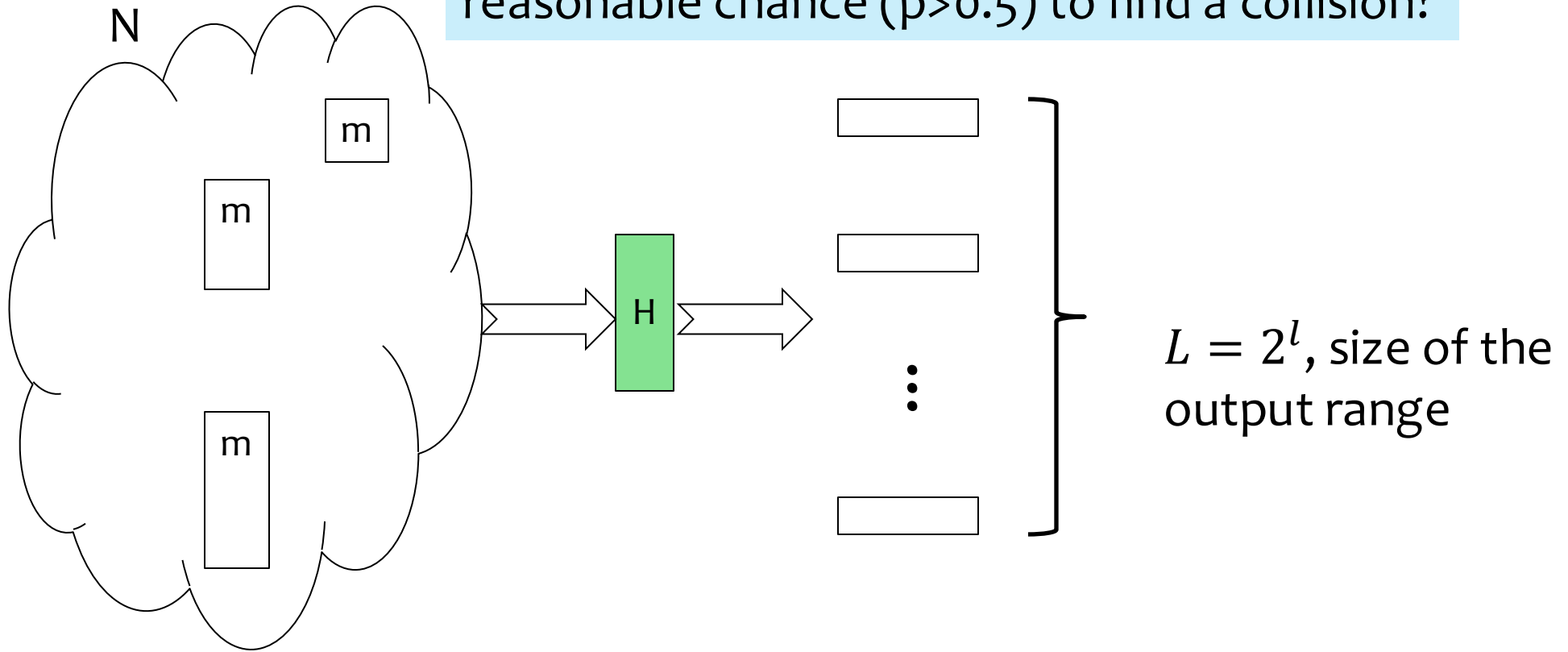
- $H: \{0,1\}^{2048} \rightarrow \{0,1\}^{256}$

Ratio of collision: $\frac{2^{l-256}}{2^l} = \frac{1}{2^{256}}$

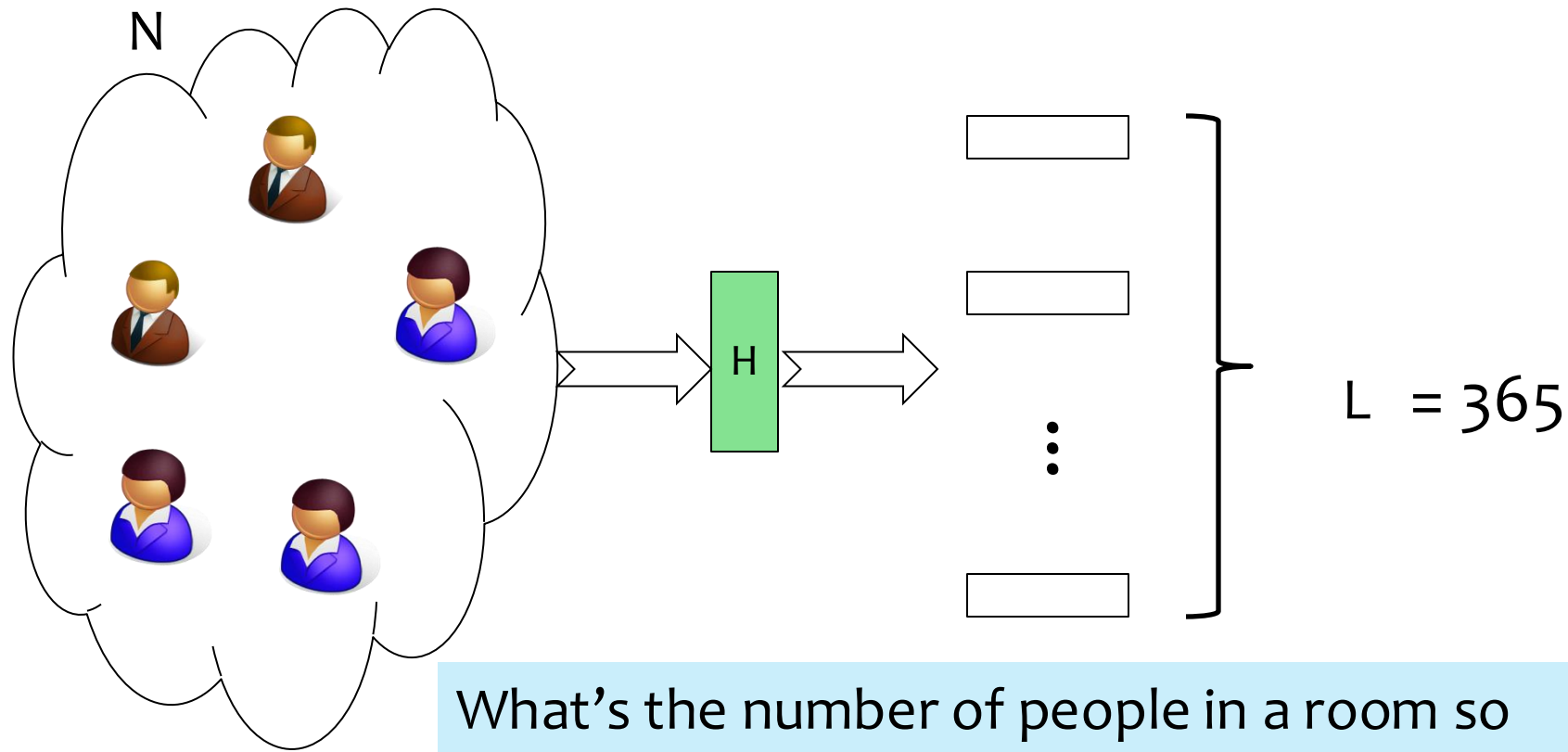
Hardness of a secure hash only depends on the **output length**, but not the input length

How hard is it to find collision?

How large does N has to be to have a reasonable chance ($p > 0.5$) to find a collision?



Birthday attack

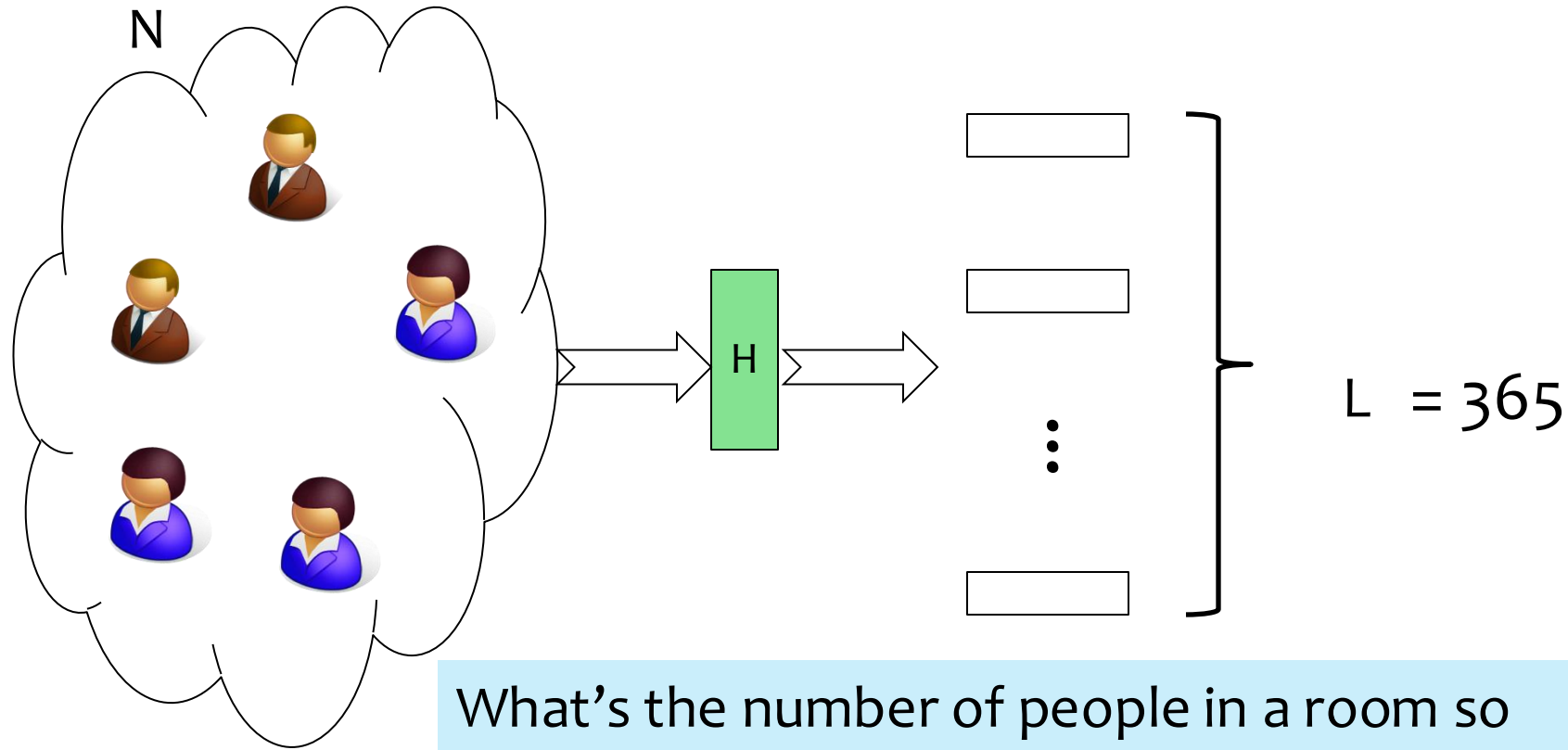


What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that two people in that room share the same birthday?

Birthday attack

$N=23$ $p=0.5$

$N=60$ $p = 0.99$



What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that two people in that room share the same birthday?

Birthday attack: proof

- Probability that all birthdays are different

$$\overline{p(N)} = 1 \times \frac{364}{365} \times \frac{363}{365} \times \dots \times \frac{365 - N + 1}{365}$$

- Probability that at least two birthdays are identical

$$p(N) = 1 - 1 \times \frac{364}{365} \times \frac{363}{365} \times \dots \times \frac{365 - N + 1}{365}$$

- Taylor approximation: $e^{-x} \approx 1 - x \Rightarrow 1 - \frac{1}{365} \approx e^{-\frac{1}{365}}$

- So: $p(N) \approx 1 - \left(e^{-\frac{1}{365}} \cdot e^{-\frac{2}{365}} \cdot \dots \cdot e^{-\frac{N-1}{365}} \right) = 1 - e^{-\frac{N(N-1)}{2 \times 365}} \approx 1 - e^{-\frac{N^2}{2 \times 365}}$

How many people needed for a collision?

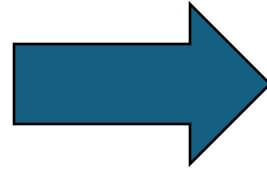
- Greater than p iff

$$1 - e^{-\frac{N^2}{2 \times 365}} > p$$

$$\Rightarrow e^{-\frac{N^2}{2 \times 365}} < 1 - p$$

$$\Rightarrow \frac{N^2}{2 \times 365} > -\ln(1 - p)$$

$$\Rightarrow N > \sqrt{2 \times 365 \ln \frac{1}{1 - p}}$$

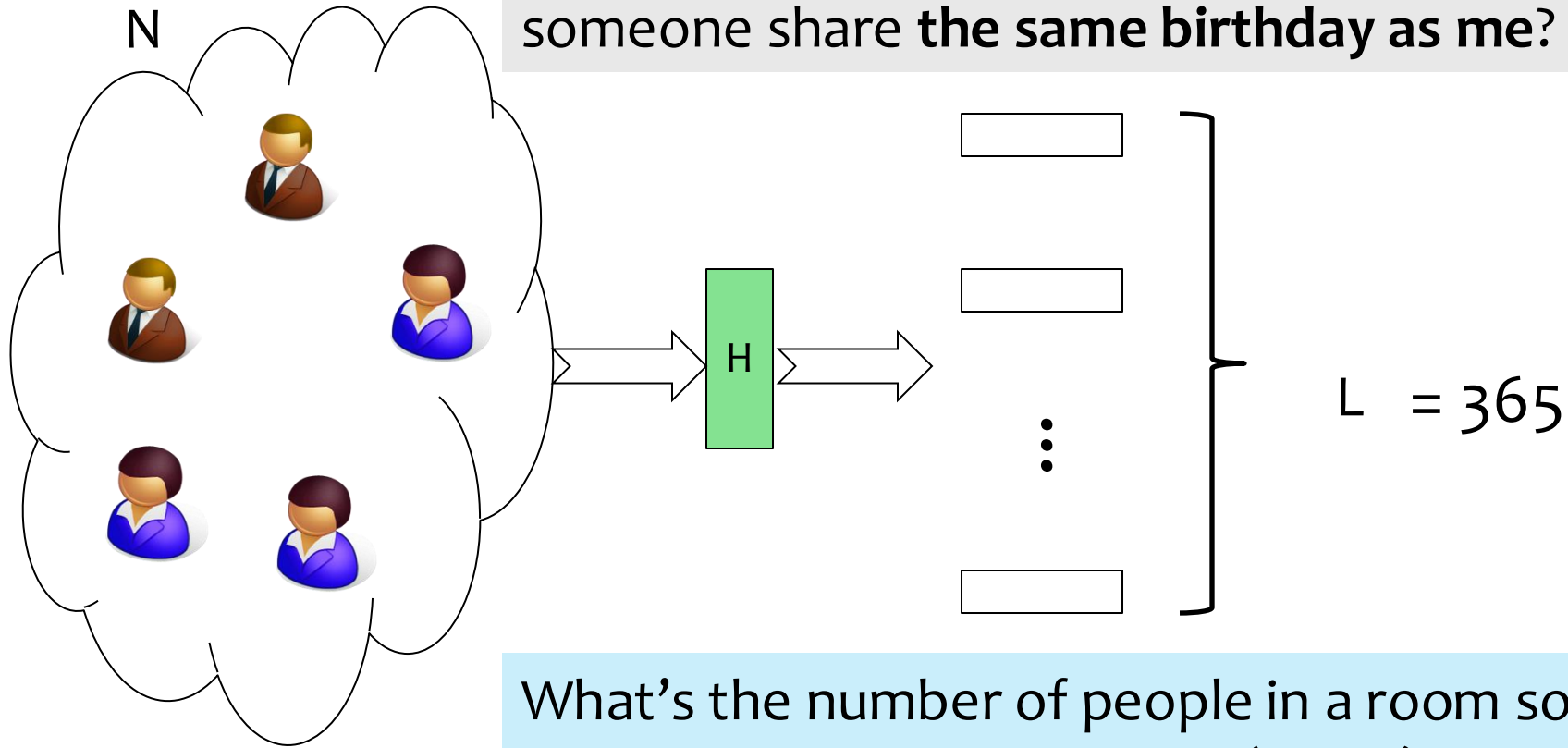


$$N > 1.17\sqrt{365} \approx 23$$

for $p = 0.5$

Why it is a "paradox"

What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that someone share **the same birthday as me?**

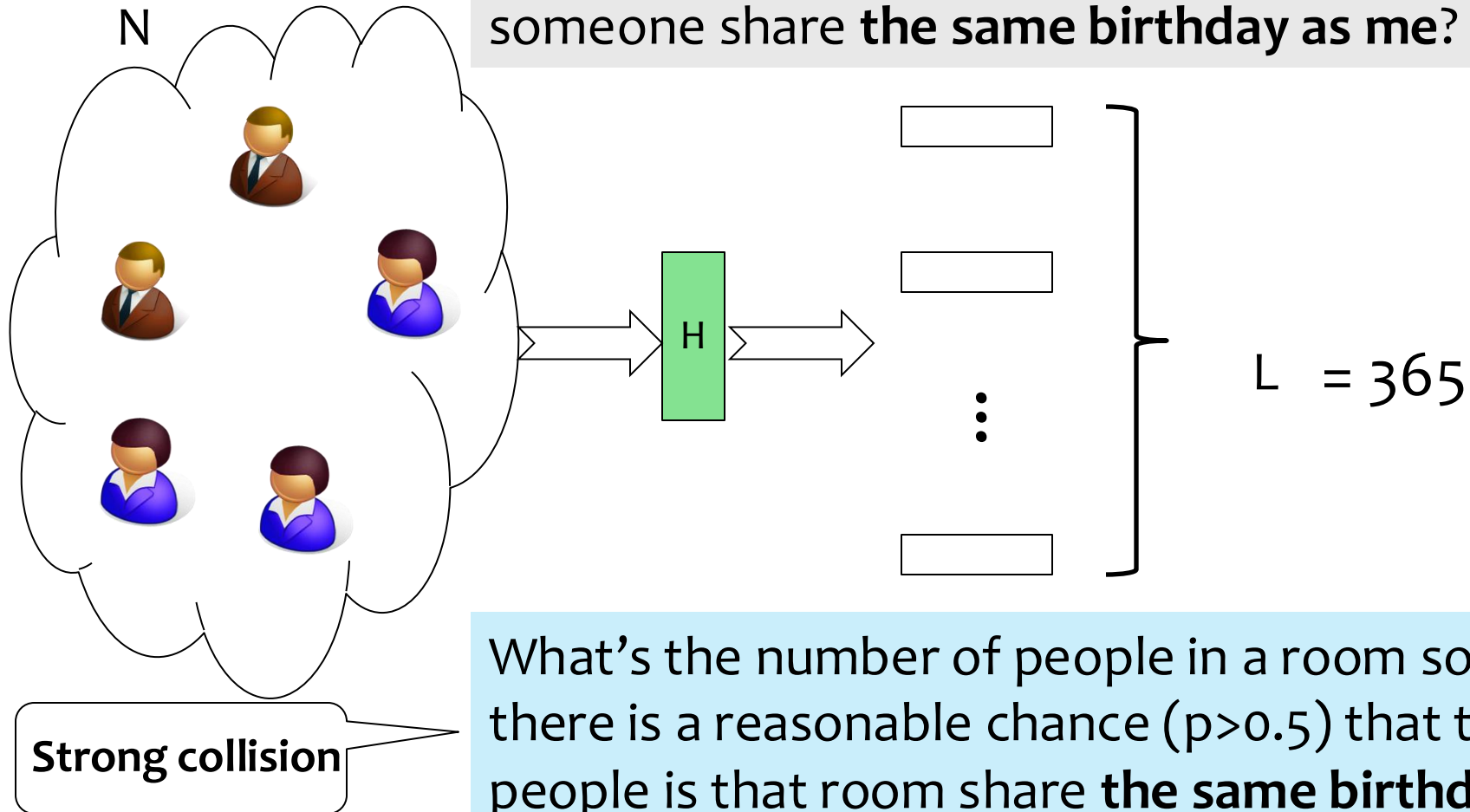


What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that two people in that room share **the same birthday?**

Why it is a paradox

Weak collision

What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that someone share **the same birthday as me?**

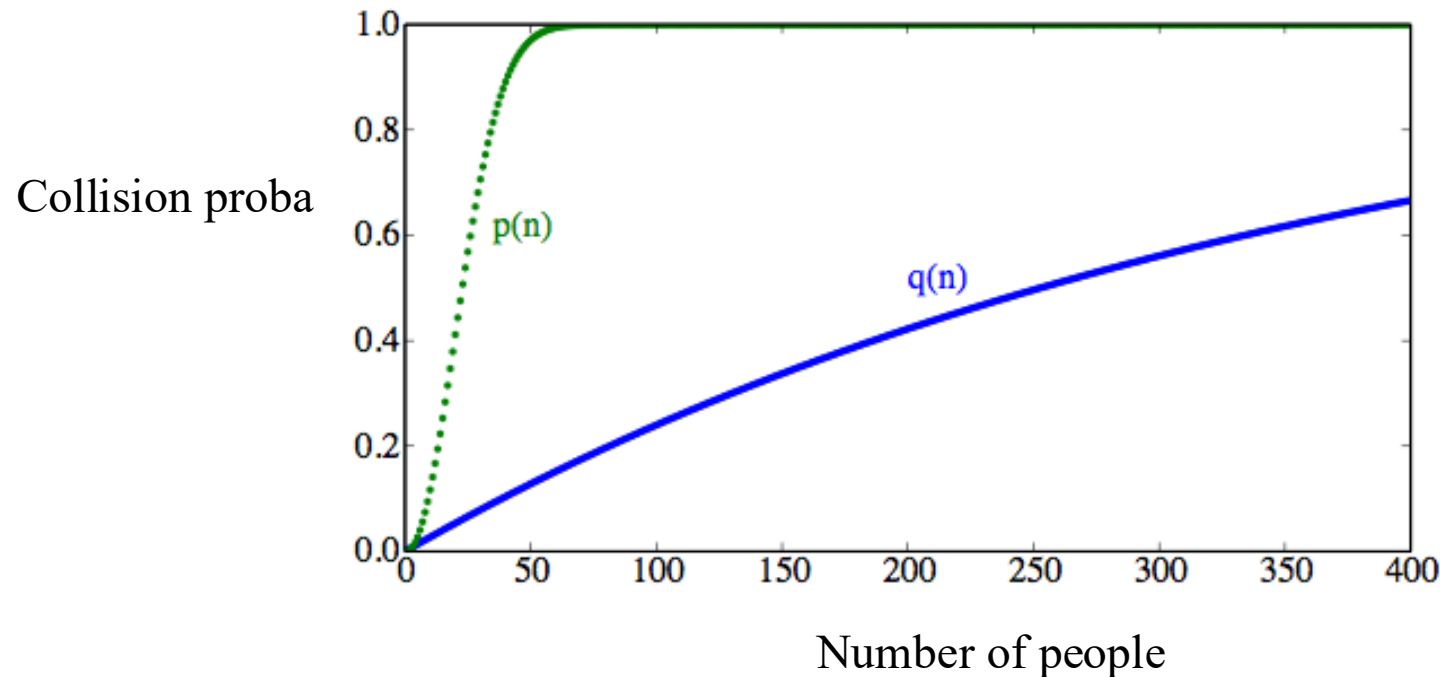


Strong collision

What's the number of people in a room so there is a reasonable chance ($p > 0.5$) that two people in that room share **the same birthday?**

Why it is a paradox

- Two same birthdays: $p(n) \approx 1 - e^{-\frac{N^2}{2 \times 365}}$
- Same birthday as me: $q(n) = 1 - \left(\frac{365-1}{365}\right)^n$



Collision resistance

- Weak collision resistant: given x , infeasible to find x' such that $H(x') = H(x)$
- Strong collision resistant: infeasible to find any x, x' such that $H(x') = H(x)$

Birthday paradox: consequences

- For any hash function mapping a number N of inputs to a number $L = 2^l$ of outputs, there is a pretty good probability of finding a collision if one can try a number of $N > \sqrt{L}$ inputs
- For the hash function to be secure
 - it must be computationally infeasible to try a number of inputs in $O(\sqrt{L})$
 - Make sure the length of the hash is large enough!

Message-Digest Algorithm 5 (MD5) -- Broken

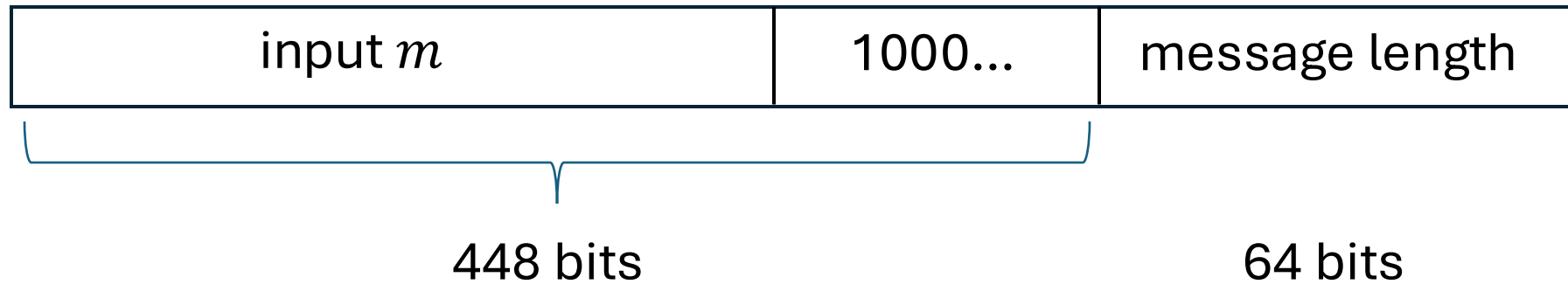
- Developed by Ron Rivest in 1991
- Uses 128-bit hash values
- How to Break MD5 and Other Hash Functions, Xiaoyun Wang and Hongbo Yu, Eurocrypt 2005 best paper award

Secure Hash Algorithm (SHA)

- Developed by NSA and approved as a federal standard by NIST
- **SHA-1** (1993)
 - 160-bits
 - Considered **insecure**
 - Still found in legacy applications
 - Vulnerabilities less severe than those of MD5
- **SHA-2 family** (2002)
 - 256 bits (SHA-256), 512 bits (SHA-512), etc.
- **SHA-3** (2015)

SHA 256

- 512-bit input to 256-bit output
- Input padding:



SHA 256

- extend



512 = 16 blocks, 32 bits each



64 blocks, 32 bits each

Extend the first 16 words into the remaining 48 words $w[16..63]$ of the message schedule array:

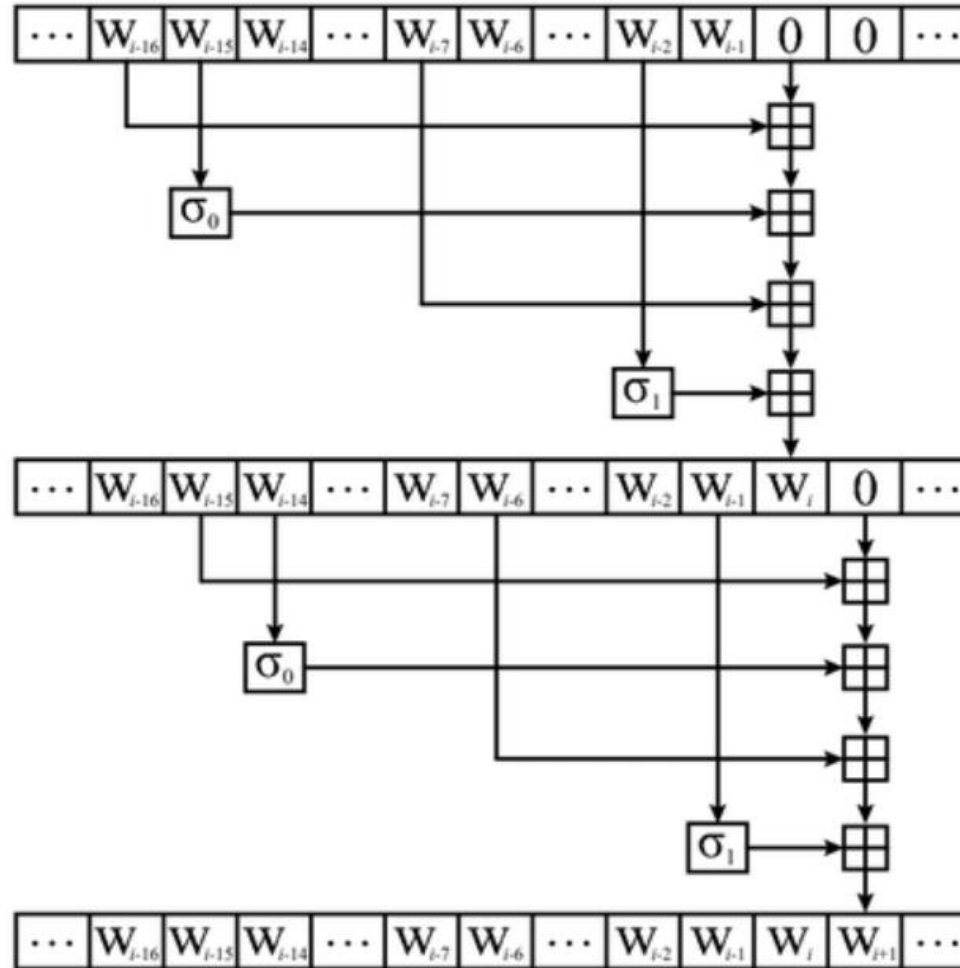
for i **from** 16 to 63

$s0 := (w[i-15] \text{ rightrotate } 7) \text{ xor } (w[i-15] \text{ rightrotate } 18) \text{ xor } (w[i-15] \text{ rightshift } 3)$

$s1 := (w[i-2] \text{ rightrotate } 17) \text{ xor } (w[i-2] \text{ rightrotate } 19) \text{ xor } (w[i-2] \text{ rightshift } 10)$

$w[i] := w[i-16] + s0 + w[i-7] + s1$

SHA 256



SHA 256 round function

- Initialize hash values: (the first 32 bits of the fractional parts of the square roots of the first 8 prime numbers)
 - $h_0 := 0x6a09e667$
 - $h_1 := 0xbb67ae85$
 - $h_2 := 0x3c6ef372$
 - $h_3 := 0xa54ff53a$
 - $h_4 := 0x510e527f$
 - $h_5 := 0x9b05688c$
 - $h_6 := 0x1f83d9ab$
 - $h_7 := 0x5be0cd19$
-
- $a := h_0, b := h_1, c := h_2, d := h_3, e := h_4, f := h_5, g := h_6, h := h_7$